

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

|                  |                                                                                                                |               |                                                |
|------------------|----------------------------------------------------------------------------------------------------------------|---------------|------------------------------------------------|
| Course Code:     | CSA06                                                                                                          | Course Title: | Design and Analysis of Algorithms              |
| CO Assessed:     | CO2 – Manipulation (BL1, BL2, BL3)                                                                             | Assessment:   | Assessment Tool 1 – Analytical Problem Solving |
| Weightage:       | 40% of CIA                                                                                                     | Total Marks:  | 100                                            |
| CO2 Statement:   | Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3) |               |                                                |
| Student Name:    | Y. Thanushma                                                                                                   | Reg. No.:     | 192425355                                      |
| Section / Batch: | 2024 / slot A / Hostel                                                                                         | Date:         |                                                |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

| Question Type               | Focus Area                                                                                         | Questions |
|-----------------------------|----------------------------------------------------------------------------------------------------|-----------|
| Application-Based Questions | Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis | Q1        |

### SECTION A – APPLICATION BASED QUESTIONS

1. Apply the Brute Force Technique using Bubble sort on dataset [11, 24, 37, 18, 29, 7].
2. Given the unsorted dataset [27, 35, 41, 53, 62, 74, 88] apply linear search to find the element 88.
3. Given the sorted dataset [4, 9, 14, 19, 24, 29, 34, 39, 44] apply binary search to find the element 34.

### Code of Conduct

I, Y. Thanushma (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Thanushma

# RUBRICS FOR CALCULATION

| Criteria                         | Wt. | Level 4 – Exemplary                                                        | Level 3 – Proficient                                  | Level 2 – Developing                           | Level 1 – Beginning                                    |
|----------------------------------|-----|----------------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------|--------------------------------------------------------|
| Understanding of Problem Context | 20% | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps                    | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts          | 30% | Accurately applies appropriate concepts to solve complex problems          | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy   | Fails to apply appropriate concepts                    |
| Problem-Solving Approach         | 20% | Logical, well-structured, and efficient approach                           | Mostly logical approach with minor gaps               | Basic approach with some inconsistencies       | Illogical or unclear approach                          |
| Accuracy of Solution             | 20% | Completely correct solution with proper justification                      | Mostly correct with minor errors                      | Partially correct solution                     | Incorrect or incomplete solution                       |
| Clarity                          | 10% | Clear, well-organized, and properly explained solution                     | Understandable with minor clarity issues              | Limited clarity and explanation                | Poorly presented and difficult to understand           |

## Marks Evaluation:

| Criteria                         | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|----------------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Understanding of Problem Context | 20% |                     |                      |                      |                     |
| Application of Concepts          | 30% |                     |                      |                      |                     |
| Problem-Solving Approach         | 20% |                     |                      |                      |                     |
| Accuracy of Solution             | 20% |                     |                      |                      |                     |
| Clarity                          | 10% |                     |                      |                      |                     |

**Total Marks (100):**

| Faculty In-charge             | Course Coordinator | Head of Department            |
|-------------------------------|--------------------|-------------------------------|
| Name: _____                   | Name: _____        | Name: _____                   |
| Signature: <u>[Signature]</u> | Signature: _____   | Signature: <u>[Signature]</u> |
| Date: <u>18/10/20</u>         | Date: _____        | Date: <u>18/10/20</u>         |



me

:- Y. Thanushma

Course Name:- Design  
and Analysis of Algorithms

Registration Number :- 192425355

Course Faculty:-

Dr. Pradeep Kumar

Course Code :- CSA0617

## CO-2 Assessment - 1

### 1) Bubble Sort using Brute Force Techniques:-

Problem Interpretation :-

Given dataset - [11, 24, 37, 18, 29, 7]

The task is to sort the numbers in ascending order using Bubble Sort Algorithm, which is a Brute Force Technique.

Bubble sort repeatedly compares adjacent elements. If the left element is greater than the right element, they are swapped. This process is repeated until no more swaps are required, meaning the array is sorted.

Initial array :- [11, 24, 37, 18, 29, 7]

There are 6 elements, so bubble sort requires 5 passes.

Pass 1 :-

Compare adjacent elements :-

$11 < 24 \rightarrow$  No swap

$24 < 37 \rightarrow$  No swap

$37 > 18 \rightarrow$  Swap

$37 > 29 \rightarrow$  Swap

$37 > 7 \rightarrow$  Swap

Array after pass 1 : [11, 24, 18, 29, 7, 37]

Comparisons = 5 ; Swaps = 3

Pass 2:-

current array

[11, 24, 18, 29, 7, 37]

Comparisons:-

$11 < 24 \rightarrow$  No swap

$24 > 18 \rightarrow$  Swap

$24 < 29 \rightarrow$  No swap

$29 > 7 \rightarrow$  Swap

Array after pass 2:- [11, 18, 24, 7, 29, 37]

comparisons = 4

swaps = 2

Pass 3:-

Current array:- [11, 18, 24, 7, 29, 37]

Comparisons:-

$11 < 18 \rightarrow$  No swap

$18 < 24 \rightarrow$  No swap

$24 > 7 \rightarrow$  Swap

Array after pass 3:- [11, 18, 7, 24, 29, 37]

Comparisons = 3 ; swaps = 1

Pass 4:-

current array:- [11, 18, 7, 24, 29, 37]

Comparisons:-

$11 < 18 \rightarrow$  No swap

$18 > 7 \rightarrow$  Swap

Array after pass 4:- [11, 7, 18, 24, 29, 37]

Comparisons = 2

Swaps = 1



Pass 5 :-

Current array :-  $[11, 7, 18, 24, 29, 37]$

Comparisons :-

$11 > 7 \rightarrow \text{swap}$

Array after pass 5 :-  $[7, 11, 18, 24, 29, 37]$

Comparisons = 1

Swaps = 1

Total Comparisons :-  $5 + 4 + 3 + 2 + 1 = 15$

Total Swaps :-  $3 + 2 + 1 + 1 + 1 = 8$

Time Complexity Analysis :-

| Case         | Time Complexity          | Explanation                                                     |
|--------------|--------------------------|-----------------------------------------------------------------|
| Best case    | $O(n)$ with optimization | Array is already sorted only one sort is needed.                |
| Average Case | $O(n^2)$                 | Requires many comparisons and some swaps                        |
| Worst Case   | $O(n^2)$                 | Array is in reverse order; maximum comparisons and swaps occur. |

Final :-  $7 < 11 < 18 < 24 < 29 < 37$

Operational Efficiency :-

Bubble sort sorts the array by repeatedly comparing and swapping adjacent elements. It is simple to implement but inefficient for large datasets due to its 15 comparisons and 8 swaps for this example.

2) Linear Search :-

Problem Interpretation :-

Given dataset :-  $[27, 35, 41, 53, 62, 74, 88]$

Find the element 88 using linear search which

checks each element sequentially from left to right until the target is found.

Initial array :-

[27, 35, 41, 53, 62, 74, 88]

Target element :- 88

Step by Step Search :-

|                 |            |         |             |
|-----------------|------------|---------|-------------|
| comparison 1 :- | compare 27 | with 88 | → Not found |
| comparison 2 :- | compare 35 | with 88 | → Not found |
| comparison 3 :- | compare 41 | with 88 | → Not found |
| comparison 4 :- | compare 53 | with 88 | → Not found |
| comparison 5 :- | compare 62 | with 88 | → Not found |
| comparison 6 :- | compare 74 | with 88 | → Not found |
| comparison 7 :- | compare 88 | with 88 | → found     |

Total comparisons :- 7 comparisons

Time Complexity Analysis :-

| Case         | Time Complexity | Explanation                           |
|--------------|-----------------|---------------------------------------|
| Best case    | $O(1)$          | Target is the first element           |
| Average Case | $O(n)$          | Target is somewhere in middle         |
| worst case   | $O(n)$          | Target is last element or not present |

Correctness Verification :-

The target element 88 is found at the 7th position (index 6) after 7 comparisons.

Therefore the search result is correct.



ational Efficiency :-

Linear Search checks each element one by one until the target is found. Since 88 is at the last position, it requires 7 comparisons, making it worst-case scenario with  $O(n)$  complexity.

### ③ Binary Search Divide and Conquer Techniques

Problem Interpretation :-

Given the sorted dataset:

[4, 9, 14, 19, 24, 29, 34, 39, 44]

The task is to find the element 34 using the Binary Search algorithms, which follows the Divide and Conquer technique.

Binary Search works only on a sorted dataset. It repeatedly compares the target with the middle element and eliminates half of the remaining search element interval, making it much faster than linear search.

Initial array :- [4, 9, 14, 19, 24, 29, 34, 39, 44]

There are 9 elements

Target element : 34

Step 1 :-

Search interval : 0 to 8

Middle Index =  $(0+8)/2 = 4$

middle element = 24

Compare 34 with 24

Since  $34 > 24$ , search the right half

New Interval :- 5 to 8

comparisons = 1

Step 2 :-

Search interval : 5 to 8

Middle Index =  $(5+8)/2 = 6$

Middle element = 34

compare 34 with 34

Target found

comparisons = 1

Summary

| Step | Search interval | middle element | Result            |
|------|-----------------|----------------|-------------------|
| 1    | 0 - 8           | 24             | Search right half |
| 1    | 5 - 8           | 34             | Found             |

Total comparisons:-  $1+1 = 2$

Time Complexity Analysis :-

| case         | Time Complexity | Explanation                                                                            |
|--------------|-----------------|----------------------------------------------------------------------------------------|
| Best Case    | $O(1)$          | Target is the middle element                                                           |
| Average Case | $O(\log n)$     | search intervals is halved at each step.                                               |
| Worst Case   | $O(\log n)$     | maximum halvings are required until the target is found or the interval becomes empty. |

Operational Efficiency :-

Binary search reduces the search interval by half in every step. It finds 34 in only 2 comparisons, making it highly efficient for sorted datasets with

$O(\log n)$  with Time complexity)